CMPT 295

Unit – Memory Hierarchy

Lecture 37 – Cache-friendly code optimization:
Locality, Memory Hierarchy and Caching

Last lecture - 1

1. Superscalar microprocessor
   - Can execute multiple machine code instructions in one clock cycle since microprocessor has multiple ***functional units*** performing similar functions -> called ***instruction-level parallelism***
   - Rearrange execution order of machine code instructions to avoid hazards (negatively impact throughput) and to maximize the use of its ***functional units*** -> called ***out of order execution***
2. Optimization
   - Software developers can restructure code: ***loop unrolling*** **(re-associating + accumulating)**
   - **Effects**:
     - Reduce number of loop iterations (loop overhead)
     - Take advantage of ***instruction-level parallelism***
   - **Conclusion**: increase throughput!

Last lecture - 2

- However, let's keep in mind that …
- Our objective as software developers is to write programs that …
  - Solve the given problems
  - Are maintainable
  - Are robust
  - And are optimized … **only if *faster performance*** is required
- Why? Because optimization …
  - Increases code length, increases possibility of introducing bugs
  - Reduces readability, modularity …
  - Optimizing programs is ***time consuming*** and ***involves trade-offs***

Today's Menu

- Cache-friendly code optimization:
  - Locality
  - Memory hierarchy
  - Caching
- Cache memories
  - How does a cache work?
  - How is cache structured?
  - Examples
  - Cache performance analysis
  - Another example of cache -> conflict miss
- Memory mountain

Goal for this unit

- Learn to optimize our code based on our understanding of
  - Locality
  - Memory hierarchy
  - Caching

Locality

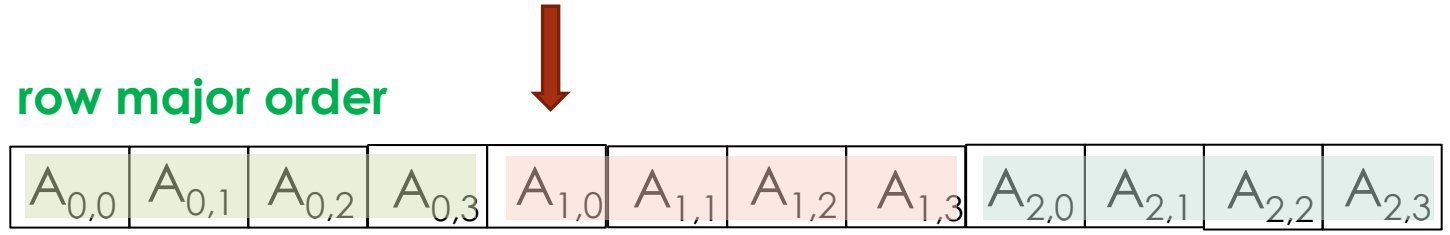
ISA - Memory model so far

Memory can be seen as …

- Linear (contiguous) array of bytes
- If the memory address resolution is a byte, then …
  - Each byte is addressable
  - Each byte is accessible in constant time
- 2D array can be viewed as:

$A_{0,0}$	$A_{0,1}$	$A_{0,2}$	$A_{0,3}$
$A_{1,0}$	$A_{1,1}$	$A_{1,2}$	$A_{1,3}$
$A_{2,0}$	$A_{2,1}$	$A_{2,2}$	$A_{2,3}$

- And in memory, it is stored as:



Do these two functions differ in their performance?

```
// M = 3, N = 4
int sumArray1(int A[M][N]) {
    int i, j, sum;
    sum = 0;
    for (i = 0; i < M; i++) {
        for (j = 0; j < N; j++)
            sum += A[i][j];
    }
    return sum;
}
```

```
// M = 3, N = 4
int sumArray2(int A[M][N]) {
    int i, j, sum;
    sum = 0;
    for (i = 0; i < N; i++) {
        for (j = 0; j < M; j++)
            sum += A[j][i];
    }
    return sum;
}
```

viewed as

$A_{0,0}$	$A_{0,1}$	$A_{0,2}$	$A_{0,3}$
$A_{1,0}$	$A_{1,1}$	$A_{1,2}$	$A_{1,3}$
$A_{2,0}$	$A_{2,1}$	$A_{2,2}$	$A_{2,3}$

in memory

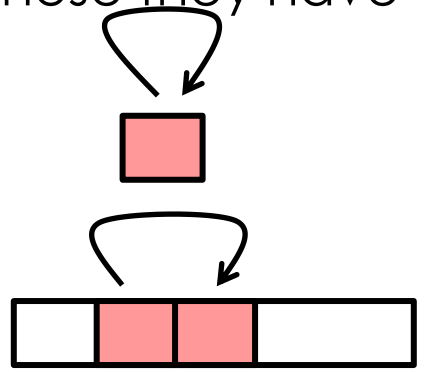
$A_{0,0}$	$A_{0,1}$	$A_{0,2}$	$A_{0,3}$	$A_{1,0}$	$A_{1,1}$	$A_{1,2}$	$A_{1,3}$	$A_{2,0}$	$A_{2,1}$	$A_{2,2}$	$A_{2,3}$

Locality

- ***Principle of Locality***
  - Programs, as they execute, tend to access (in memory) either
    - the same data or the same instructions they have recently accessed

    OR
    - data or instructions that are near (in memory) to those they have recently accessed

as a program executes …

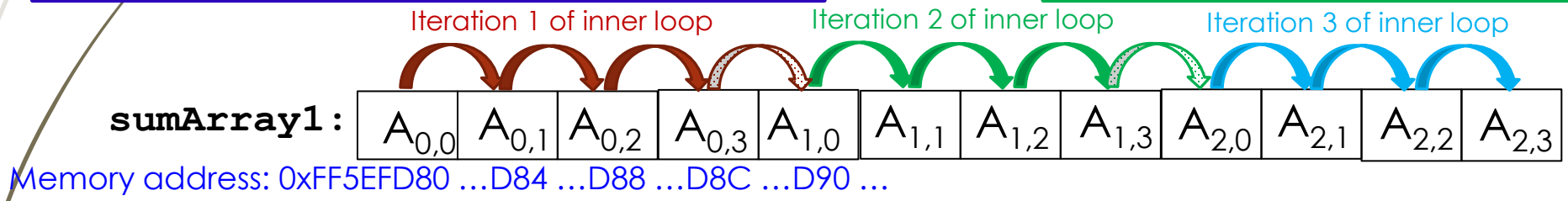


- ***Temporal locality***
  - Recently accessed data or instructions are likely to be accessed again in the near future
- ***Spatial locality***
  - Data or instructions located close by in memory (i.e., with nearby memory addresses) tend to be accessed close together in time
- ***Key skill for s/w developers***
  - Being able to look at code and get a *qualitative* sense of its *locality*

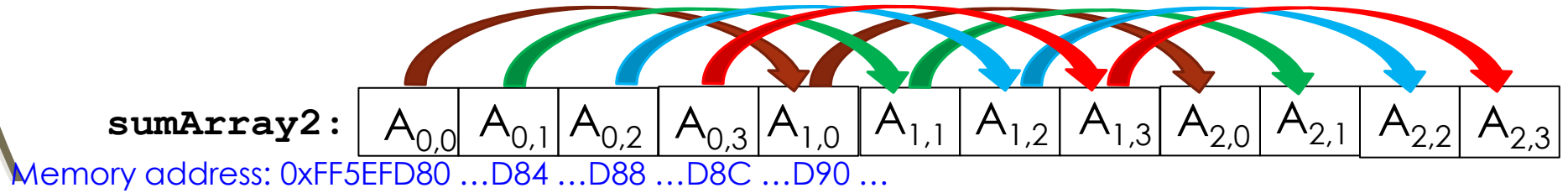
Locality and stride

```
// M = 3, N = 4
int sumArray1(int A[M][N]) {
    int i, j, sum;
    sum = 0;
    for (i = 0; i < M; i++) {
        for (j = 0; j < N; j++)
            sum += A[i][j];
    }
    return sum;
}
```

```
// M = 3, N = 4
int sumArray2(int A[M][N]) {
    int i, j, sum;
    sum = 0;
    for (i = 0; i < N; i++) {
        for (j = 0; j < M; j++)
            sum += A[j][i];
    }
    return sum;
}
```



Stride-1: each iteration of inner loop accesses an element **1 element** away from previous element



Stride-N: each iteration of inner loop accesses an element **N elements** away from previous element

Conclusion: good or bad locality?

```
// M = 3, N = 4
int sumArray1(int A[M][N]) {
    int i, j, sum;
    sum = 0;
    for (i = 0; i < M; i++) {
        for (j = 0; j < N; j++)
            sum += A[i][j];
    }
    return sum;
}
```

```
// M = 3, N = 4
int sumArray2(int A[M][N]) {
    int i, j, sum;
    sum = 0;
    for (i = 0; i < N; i++) {
        for (j = 0; j < M; j++)
            sum += A[j][i];
    }
    return sum;
}
```

- Function **sumArray1** has good or bad spatial locality with respect to array `A`
- Function **sumArray2** has good or bad spatial locality with respect to array `A`
- Function **sumArray1** has good or bad temporal locality with respect to `sum`
- Function **sumArray2** has good or bad temporal locality with respect to `sum`

Another example of ***temporal locality***
(within the execution of 1 iteration)

```
int str_alnum3(char *s) {
    int i;
    int len = strlen(s);
    for (i = 0; i < len; i++) {
        if (!(
        ((s[i] >= 'a') && (s[i] <= 'z')) ||
        ((s[i] >= 'A') && (s[i] <= 'Z')) ||
        ((s[i] >= '0') && (s[i] <= '9')))) {
            return 0;
        }
    }
    return 1;
}
```

```
int str_alnum4(char *s) {
    int i;
    int len = strlen(s);
    char aChar;
    for (i = 0; i < len; i++) {
        aChar = s[i];
        if (!(
        ((aChar >= 'a') && (aChar <= 'z')) ||
        ((aChar >= 'A') && (aChar <= 'Z')) ||
        ((aChar >= '0') && (aChar <= '9')))) {
            return 0;
        }
    }
    return 1;
}
```

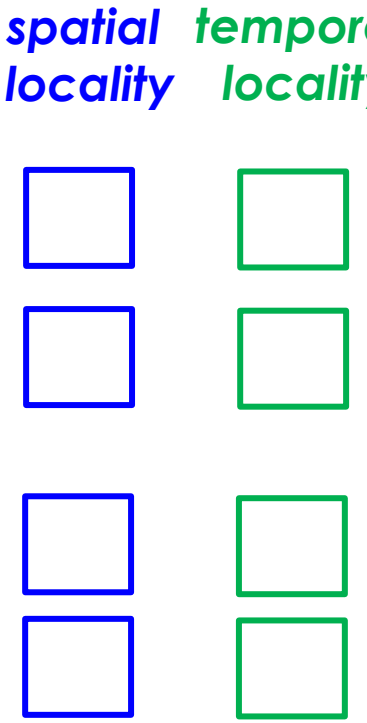
Also, an example of ***spatial locality***
(accessing adjacent letters in the string at each iteration of the loop)

Let's try!

```
total = 0;
for (i = 0; i < n; i++)
   total += B[i];
return total;
```

The statements below describe the code above

- Data access
  - *Array elements are accessed in succession* (stride-1 access pattern)
  - *Variable* `total` *is accessed at each iteration*
- Instruction access
  - *Instructions are accessed in sequence*
  - *Loop is cycled through repeatedly*
- Which ones are examples of ***spatial locality*** and which ones are examples of ***temporal locality***?



Locality-based code optimization

- Understanding ***temporal*** and ***spatial locality*** allows us to optimize our code, i.e., write code with ***good locality***, which tends to run faster
- Let's see why!

Memory Hierarchy
&
Caching

Microprocessor versus ***memory*** performance – The gap

- Section 6.1 (Storage Technologies) tells us that …
  - Access time of storage technologies has improved over the years but not as much as microprocessor clock cycle time

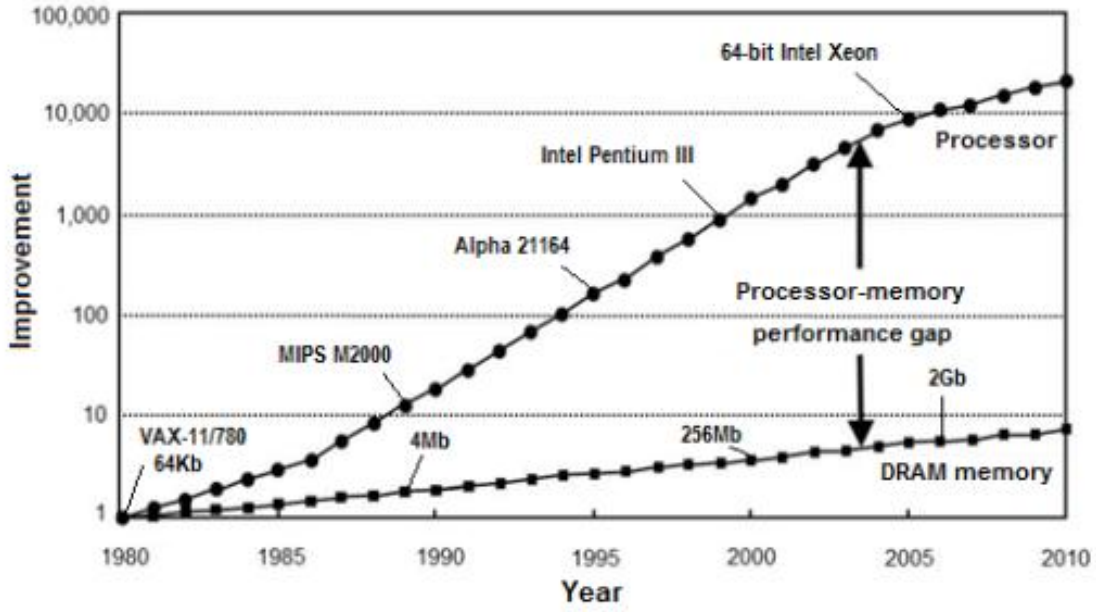


Figure 2. Yearly improvement of processor and DRAM memory speeds, for a time period of three decades.

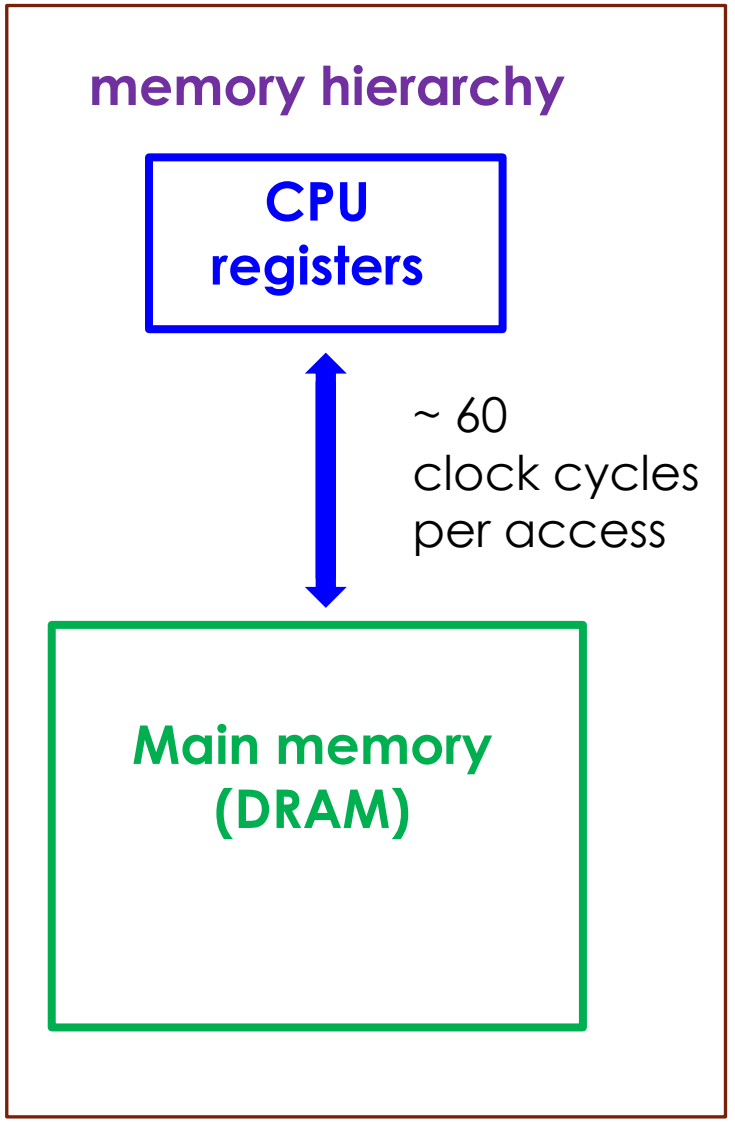
Source:

D. Efnusheva, A. Cholakoska, A. Tentov, A survey of different approaches for overcoming the processor—memory bottleneck. Int. J. Comput. Sci. Inf. Technol. 9(2), 151–163 (2017)

https://link.springer.com/chapter/10.1007/978-3-030-18338-7_15

Our view of ***memory*** so far …

- So far, our view of *memory* comprises of microprocessor registers (register file) and main memory (RAM):
  - Size
    - microprocessor registers: ~KB
    - main memory: ~GB
  - Access time
    - microprocessor registers: 0.33ns
    - main memory: 20ns



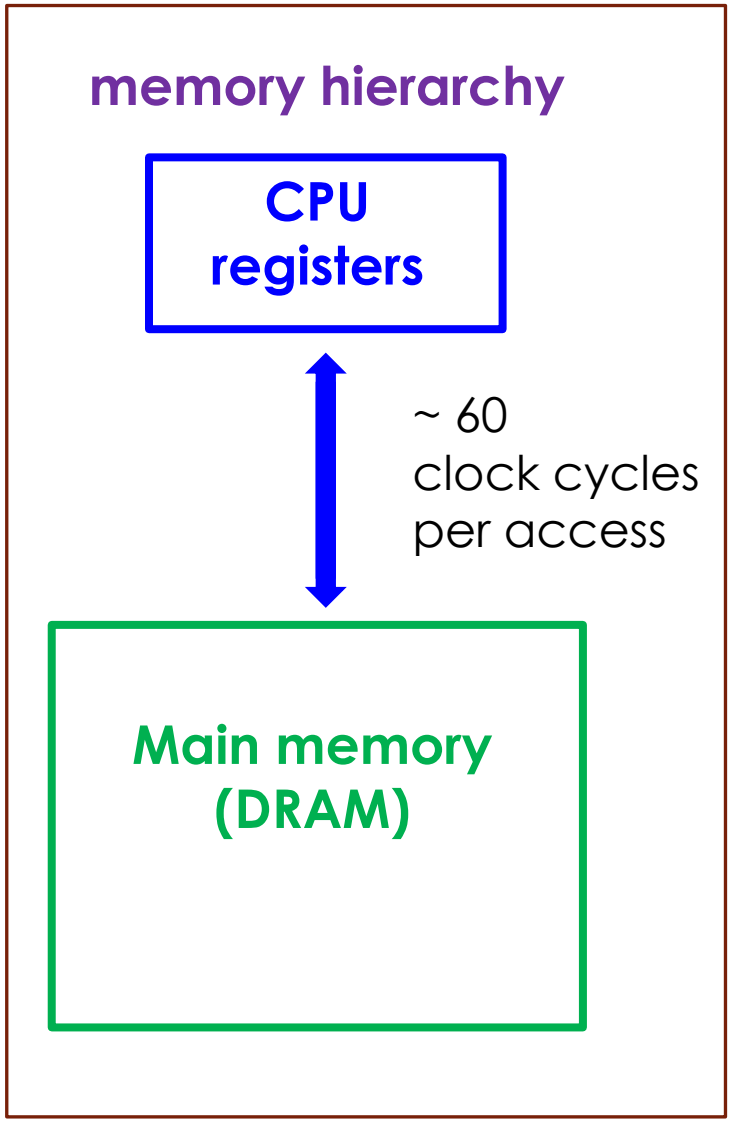
Source: From Figure 6.15 from our textbook

What does this ***gap*** mean?

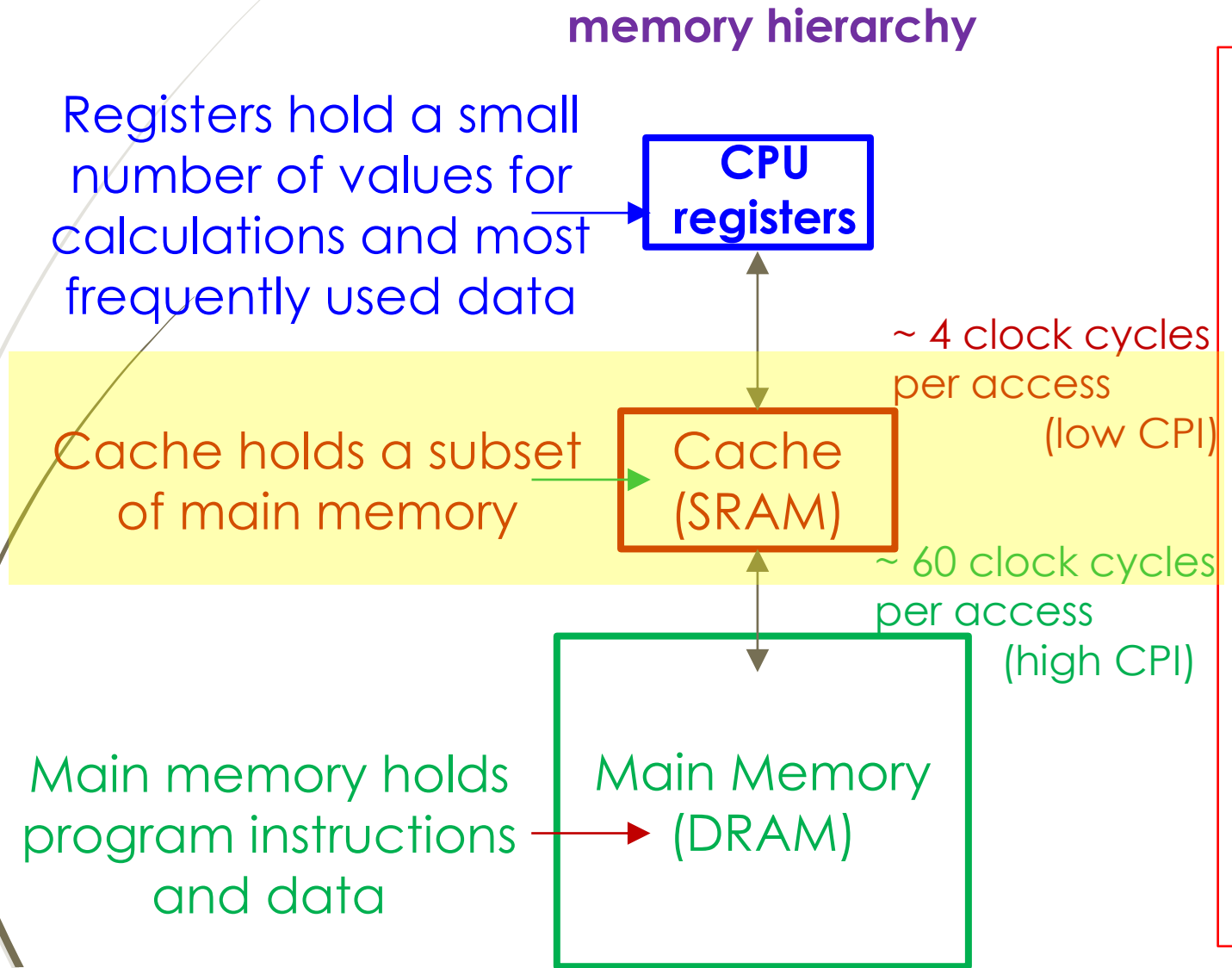
- Here is a typical scenario, considering our view of the *memory*:

1. When the microprocessor needs data (or instructions)
2. If this data is not found in one of its registers then microprocessor requests this data (1 word size of data) from main memory
3. Main memory (large and slow) takes ~60 cycles to send requested data to microprocessor
   - In the meantime, microprocessor is idling

- Over the years, as the gap grew, the amount of time the microprocessor spent waiting for main memory to deliver data grew as well
  - Impractical



Solution to the gap:
Expand memory hierarchy by introducing ***cache***



Cache holds several lines, each 64 bytes

- each line can hold a copy of 64 bytes from main memory (or be *empty* i.e., invalid)

Microprocessor interacts with cache instead of main memory

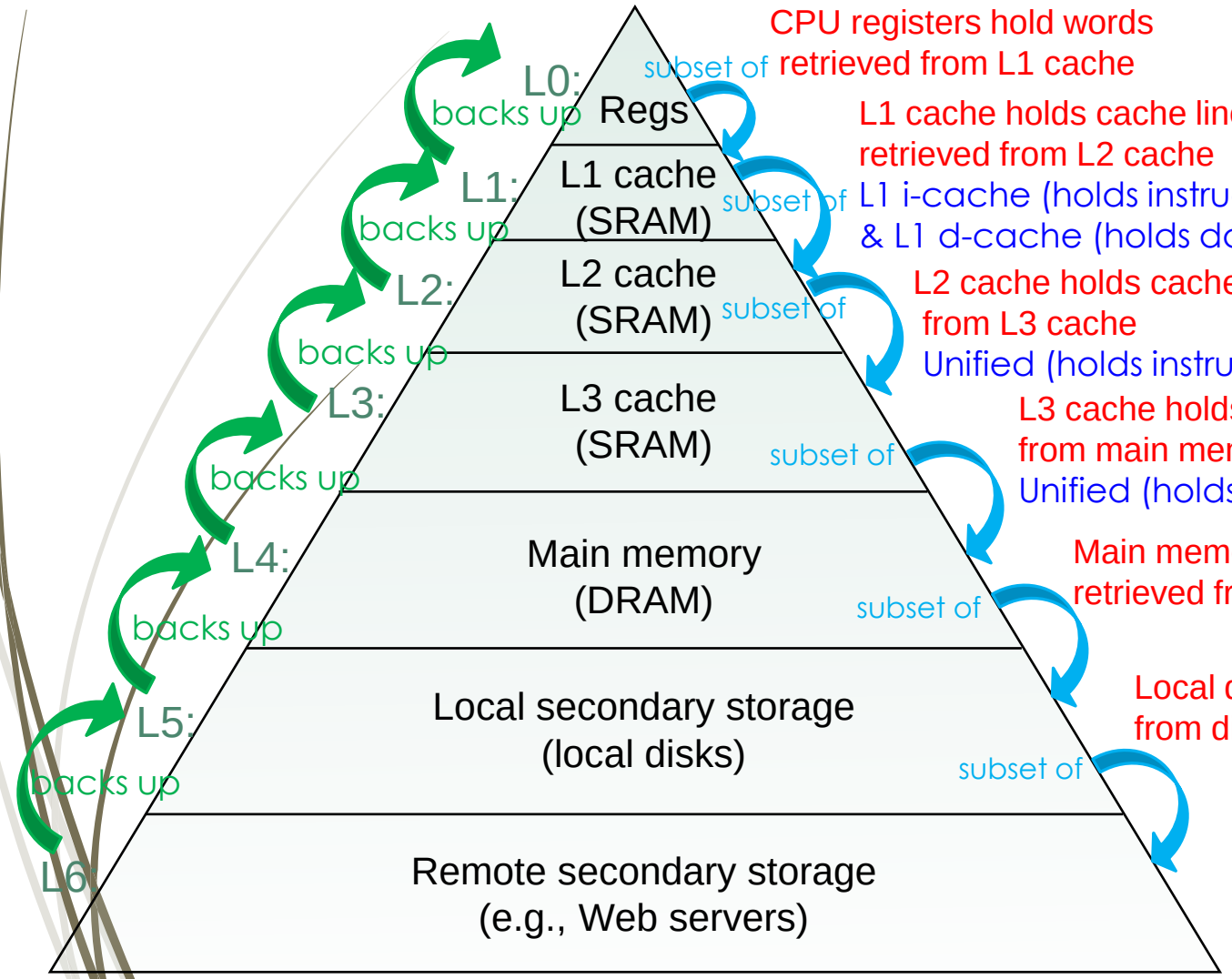
- if line in cache, data is *served* (**cache hit**)
- if line is not in cache, cache must load it from main memory (**cache miss**)

Cache

Why is it called ***cache***?
- From the French verb ***cacher***, which means ***to hide***
- Cache is ***a hidden storage place***
  - Example: food cache

- DRAM is big, but slow
  - waiting 60 cycles per access is impractical
  - both fetch (**F**) & memory (**M**) stages would have to be expanded
- Strategy: Cache
  - Add a layer to memory hierarchy -> using smaller, faster memory
    - Store frequently used data and instructions
  - Takes advantage of locality principle:
    - When data or instructions are accessed often, they are copied into cache
    - ***Temporal locality***: recent memory accesses will be needed again soon
    - ***Spatial locality***: memory nearby recent memory accesses will be needed soon

Memory hierarchy

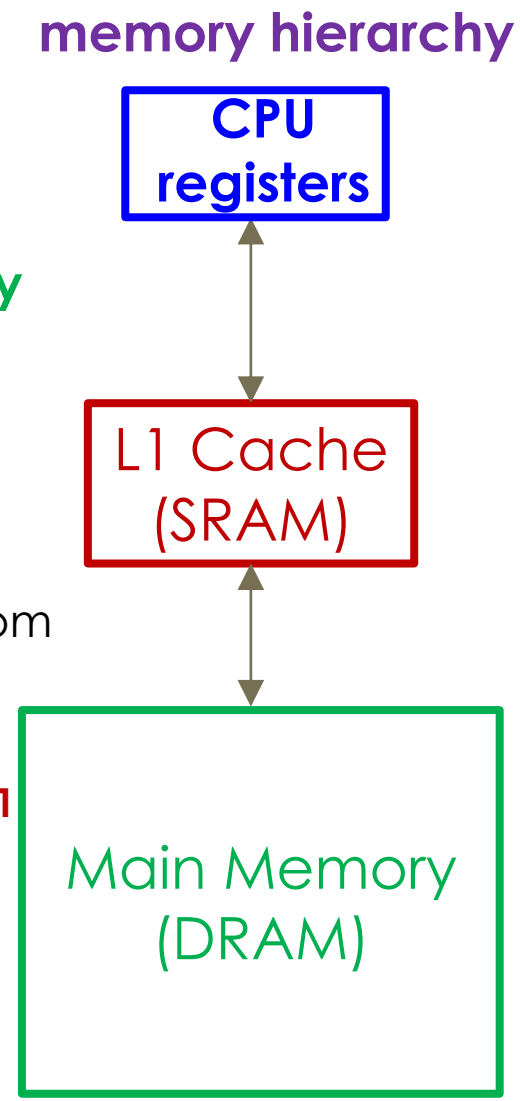


	Latency (clock cycles per access)	Size	
CPU registers hold words retrieved from L1 cache	1	1 KB	Faster, more expensive (per byte), smaller
L1 cache holds cache lines retrieved from L2 cache L1 i-cache (holds instructions) & L1 d-cache (holds data)	4	32 KB	
L2 cache holds cache lines retrieved from L3 cache Unified (holds instructions and data)	10	256 KB	
L3 cache holds cache lines retrieved from main memory Unified (holds instructions and data)	40-75	8 MB	Storage devices
Main memory holds disk blocks retrieved from local disks	200	16 GB	
Local disks hold files retrieved from disks on remote servers	M's	TB's	Slower cheaper (per byte), larger

Source: Figure 6.21 from Chapter 6 – Section 6.3 of our textbook

How does cache work? In a nutshell!

- When microprocessor executes a **LOAD** instruction
  - **LOAD a, rC** -> *read data from memory at memory address* **a**
  - Microprocessor actually reads the content of memory address **a** from **L1 cache** instead of from **main memory**
  - If content of memory address **a** is in **L1 cache**
    - Cache hit and content of memory address **a** is sent to microprocessor
  - If content of memory address **a** is not in **L1 cache**
    - Cache miss and content of memory address **a** is read from the next level down, i.e., from **main memory**
    - If content of memory address **a** is in **main memory**
      - Cache hit and content of memory address **a** is copied to **L1 cache** and then sent to microprocessor
    - If content of memory address **a** is not in main memory
      - Cache miss and content of memory address **a** is read from the next level down …



Summary

- Over the years, the performance of microprocessors has been increasing faster than memory performance (time it takes to access memory) creating a growing **gap** between them
  - This gap represents the large time difference between ***accessing registers*** and **accessing main memory**
- To narrow this gap, a **cache** is added to memory forming a ***memory hierarchy***
  - **Cache** uses faster memory components (SRAM: 4 clock cycles per access) to hold copy of data/instructions held in main memory, data/instructions which is likely to be used in near future, i.e., in the following memory accesses
  - **Cache** takes advantage of ***locality***
    - ***Temporal locality***: access the same memory location again soon
    - ***Spatial locality***: access a nearby memory location soon
- Understanding ***locality***, ***memory hierarchy*** and ***caching*** allows us to write ***cache-friendly code***
  - Programs with ***good locality*** tend to run faster because such code maximizes the use of data/instructions found in cache by accessing same memory location again and again (***temporal locality***) and/or by accessing nearby locations held in cache (***spatial locality***)

Next Lectures

- Cache-friendly code optimization:
  - Locality
  - Memory hierarchy
  - Caching
- Cache memories
  - How does a cache work?
  - How is cache structured?
  - Examples
  - Cache performance analysis
  - Another example of cache -> conflict miss
- Memory mountain